

Example of software change

- Application framework Drawlets
 - adds graphical display to a host application
- Drawing canvas
 - lines, free-hand lines, rectangles, rounded rectangles, triangles, pentagons, polygons, ellipses, text boxes, images

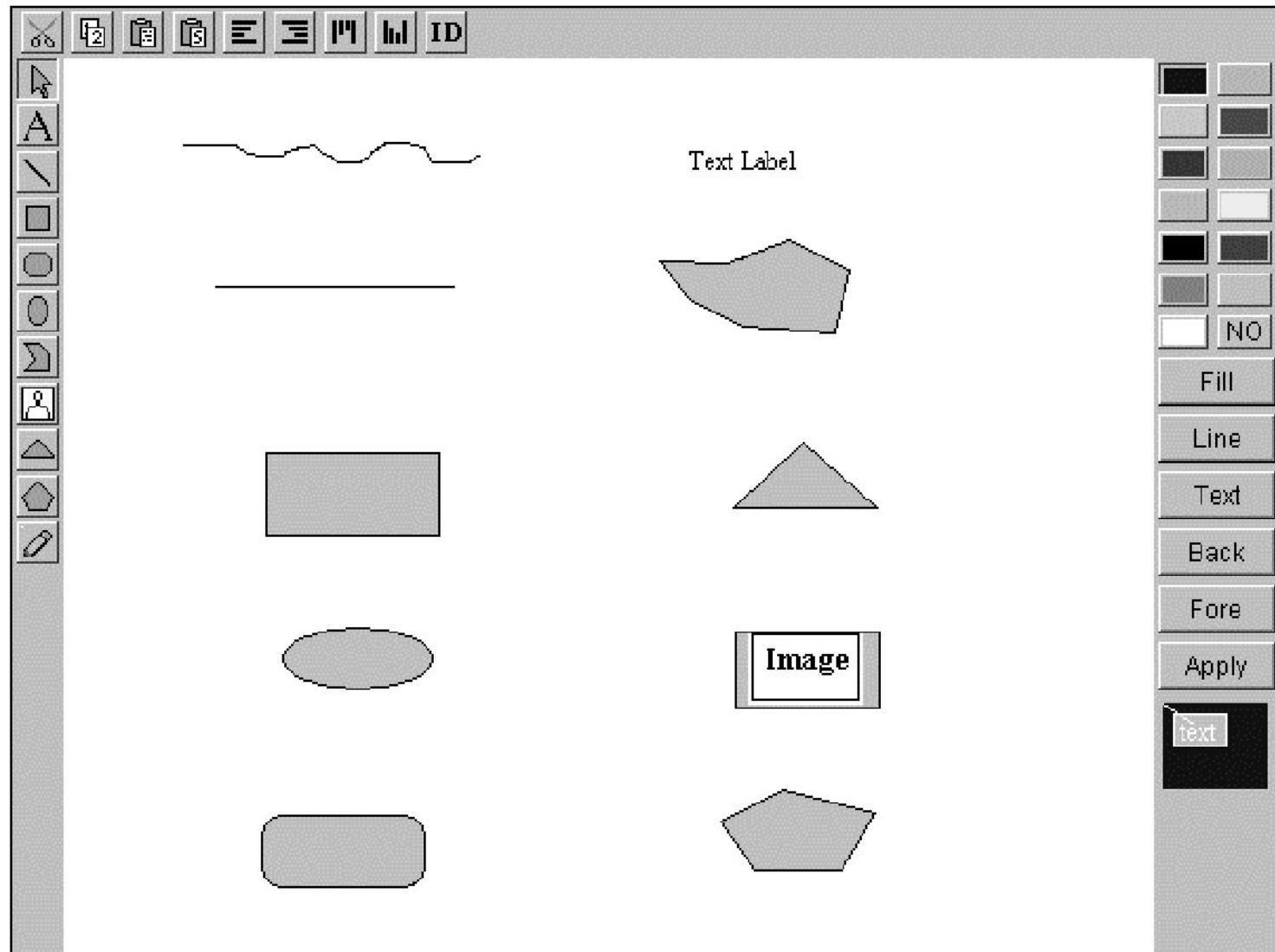
Drawlets - features

- More than 100 classes, 35 interfaces and 40,000 lines of code
 - originally implemented by Kent Beck, Ward Cunningham
 - later ported into Java
 - “perfect API”
 - <http://www.rolemodelsoft.com/aboutUs/drawlets.htm>

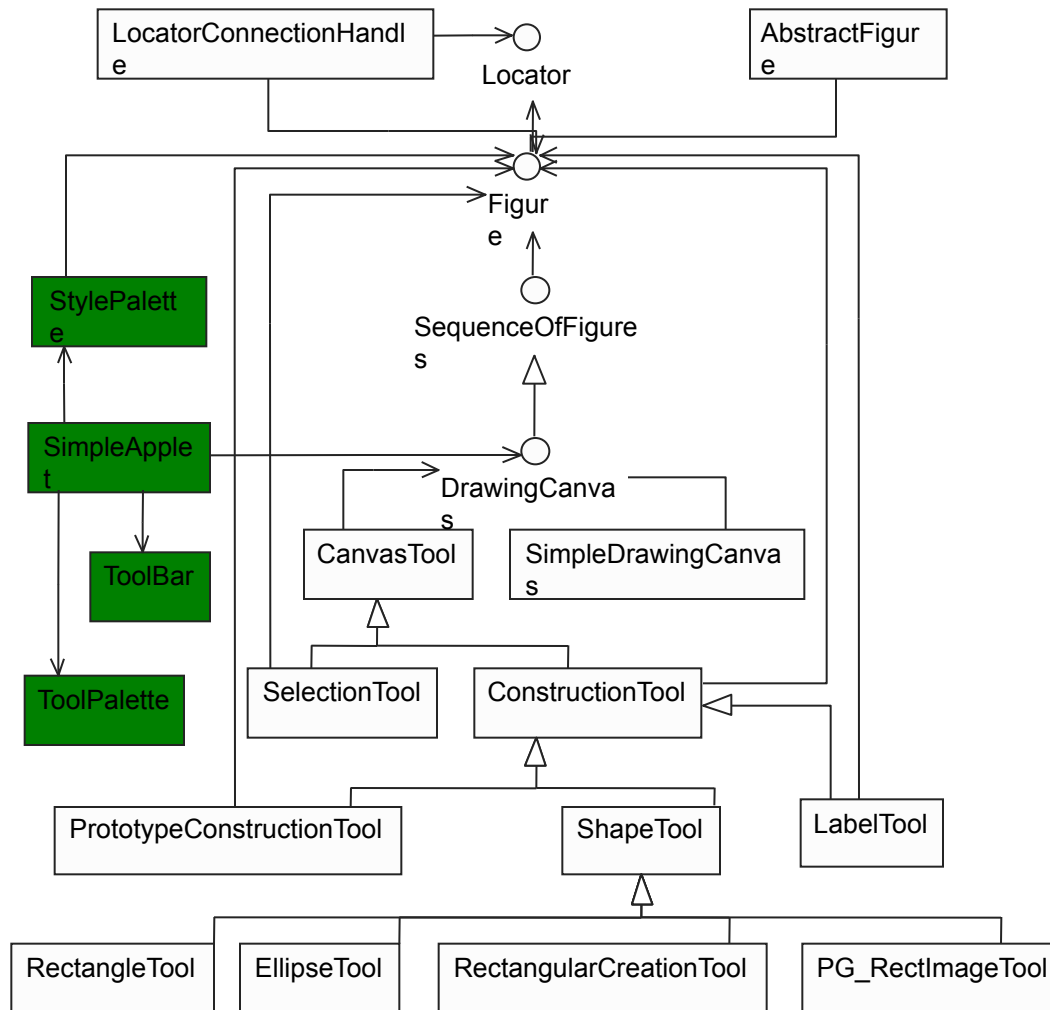
The host application

- Responsible for providing an instance of the drawing canvas, toolbars, and tool buttons
- Class SimpleApplet is a host application
 - is a part of the Drawlet library
 - SimpleApplet runs in any browser

SimpleApplet window



Top classes



Change Request

- Implement an owner for each figure.
 - An owner is the user who put the figure onto the canvas, and only the owner should be allowed to modify it.
 - At the beginning of a session, the users input their ID and password and they are the owners of all figures that were created during the session.
- This change will make SimpleApplet more versatile and useful
 - support for cooperative work

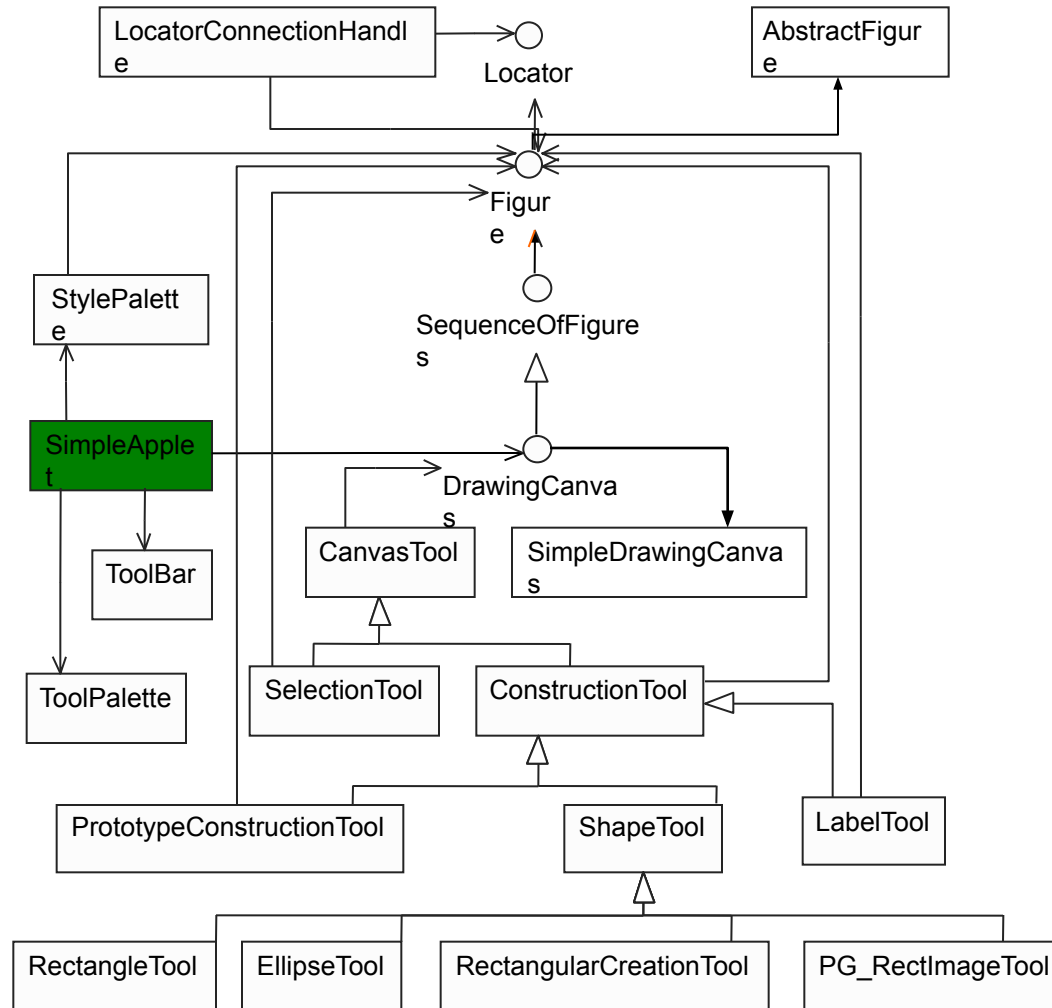
Concepts

- Implement an owner for each figure.
 - An owner is the user who put the figure onto the canvas, and only the owner should be allowed to modify it.
 - At the beginning of a session, the users input their ID and password and they are the owners of all figures that were created during the session

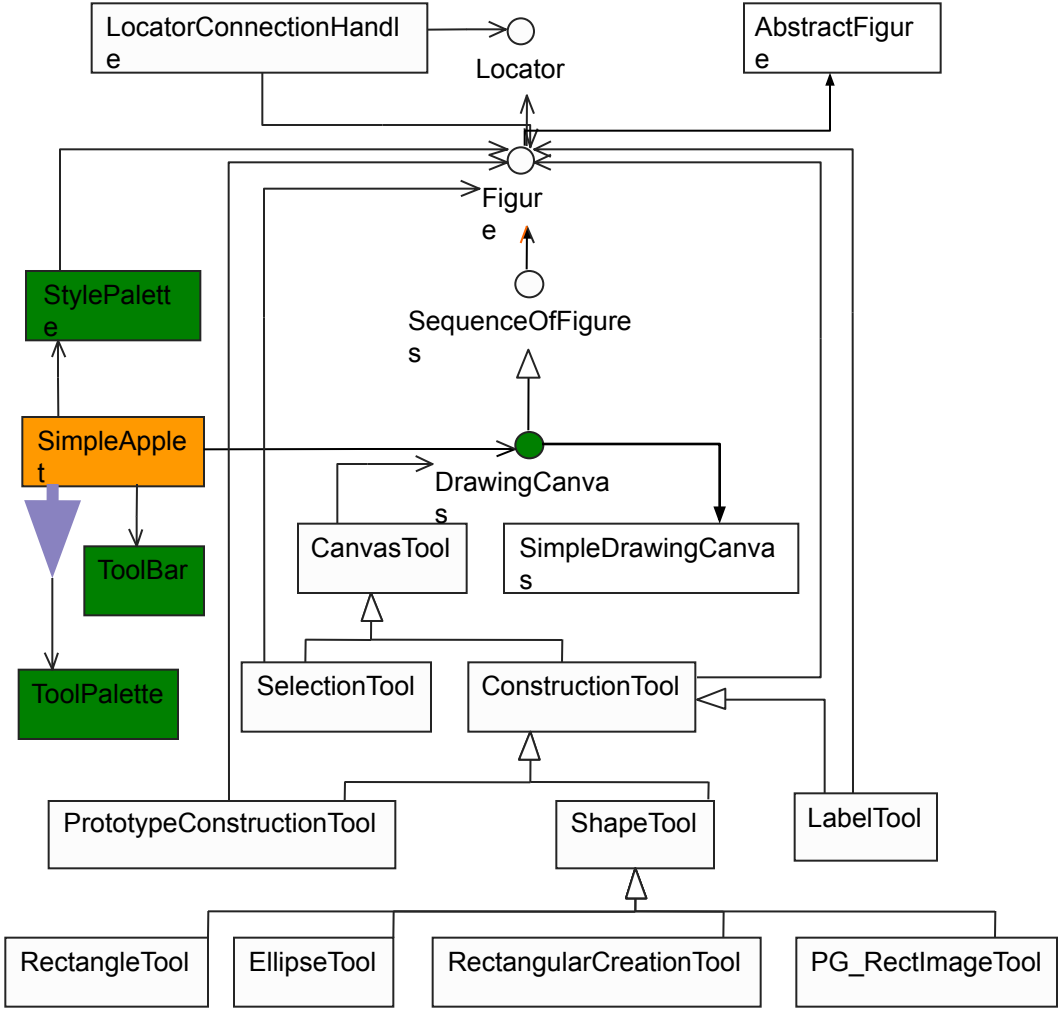
Concept classification

	irrelevant	external	significant
implement	x		
owner		x	
figure			x
user	x		
canvas			x
allowed	x		
modify	x		
beginning	x		
session			
input	x		
ID		x	
password		x	
created	x		

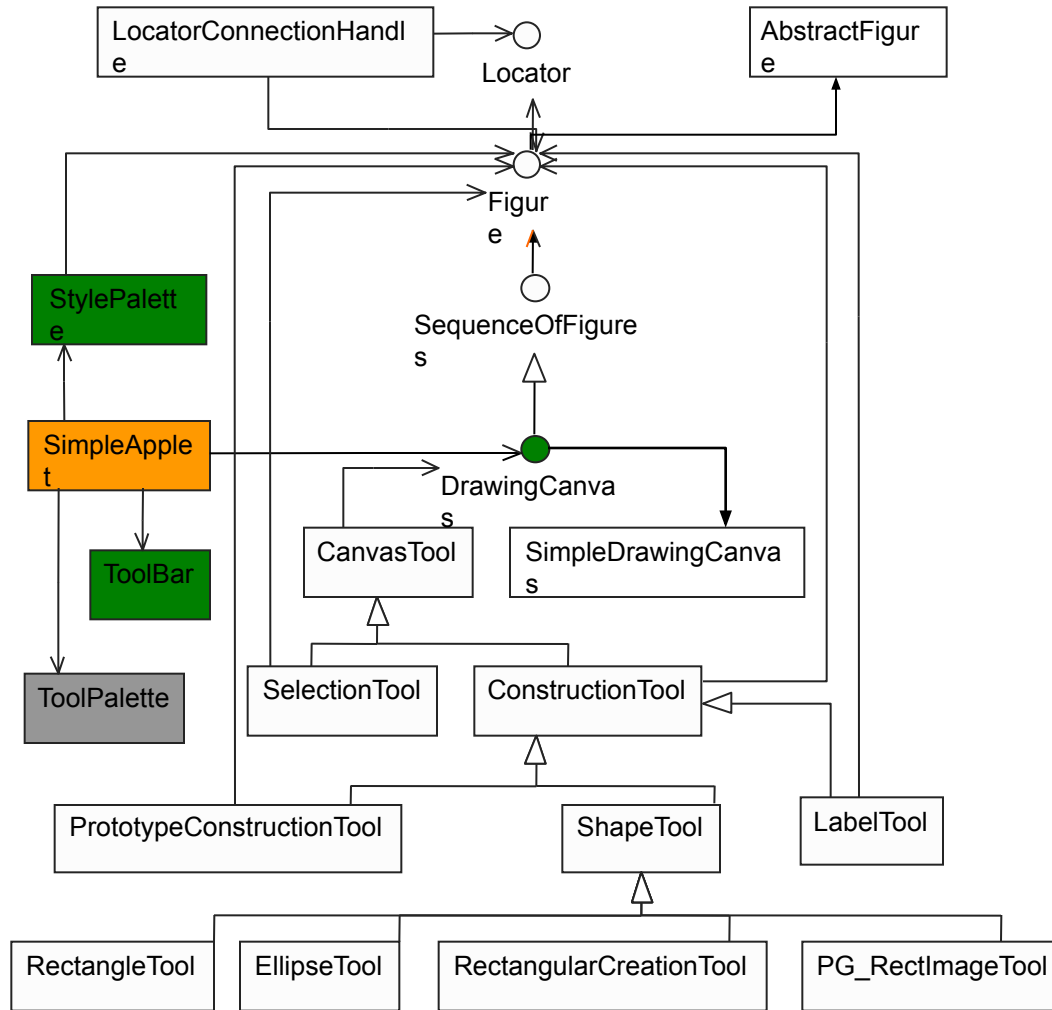
Locating figure properties



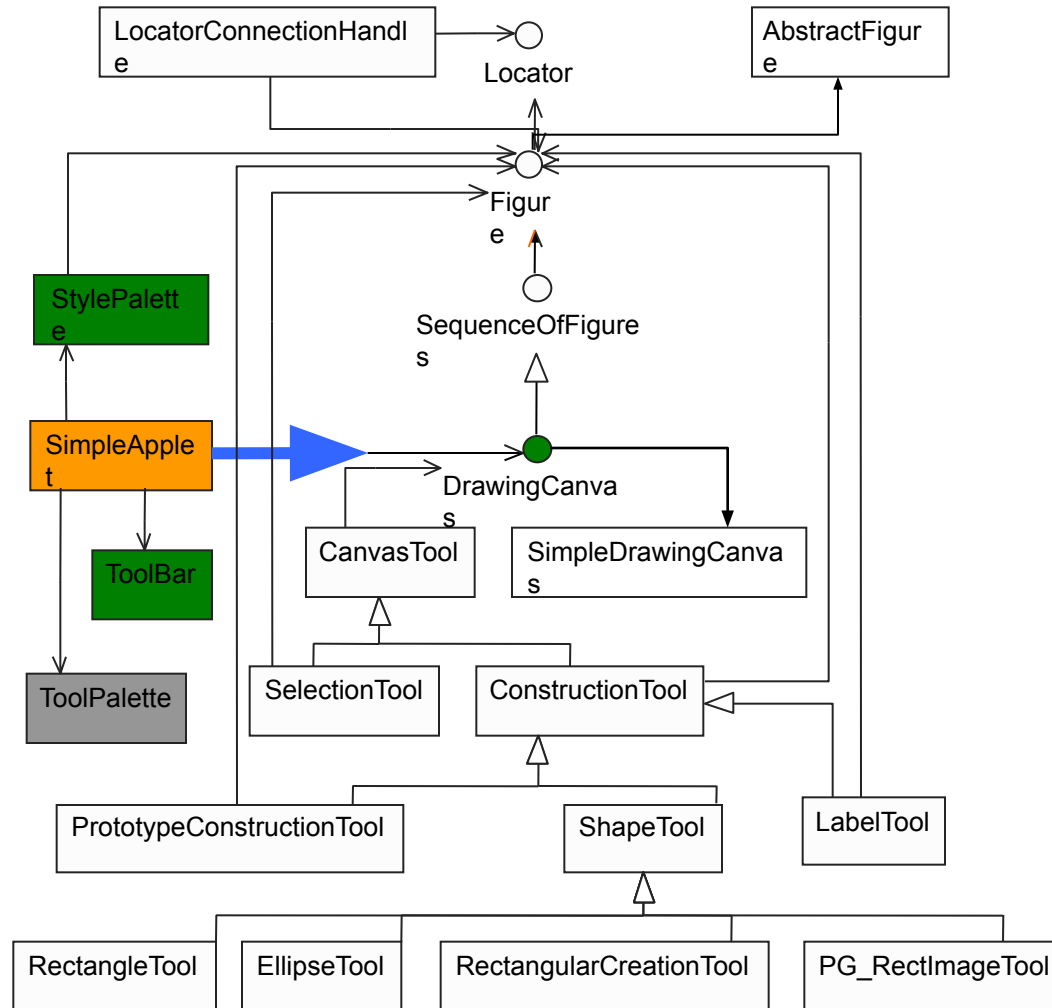
Wrong way



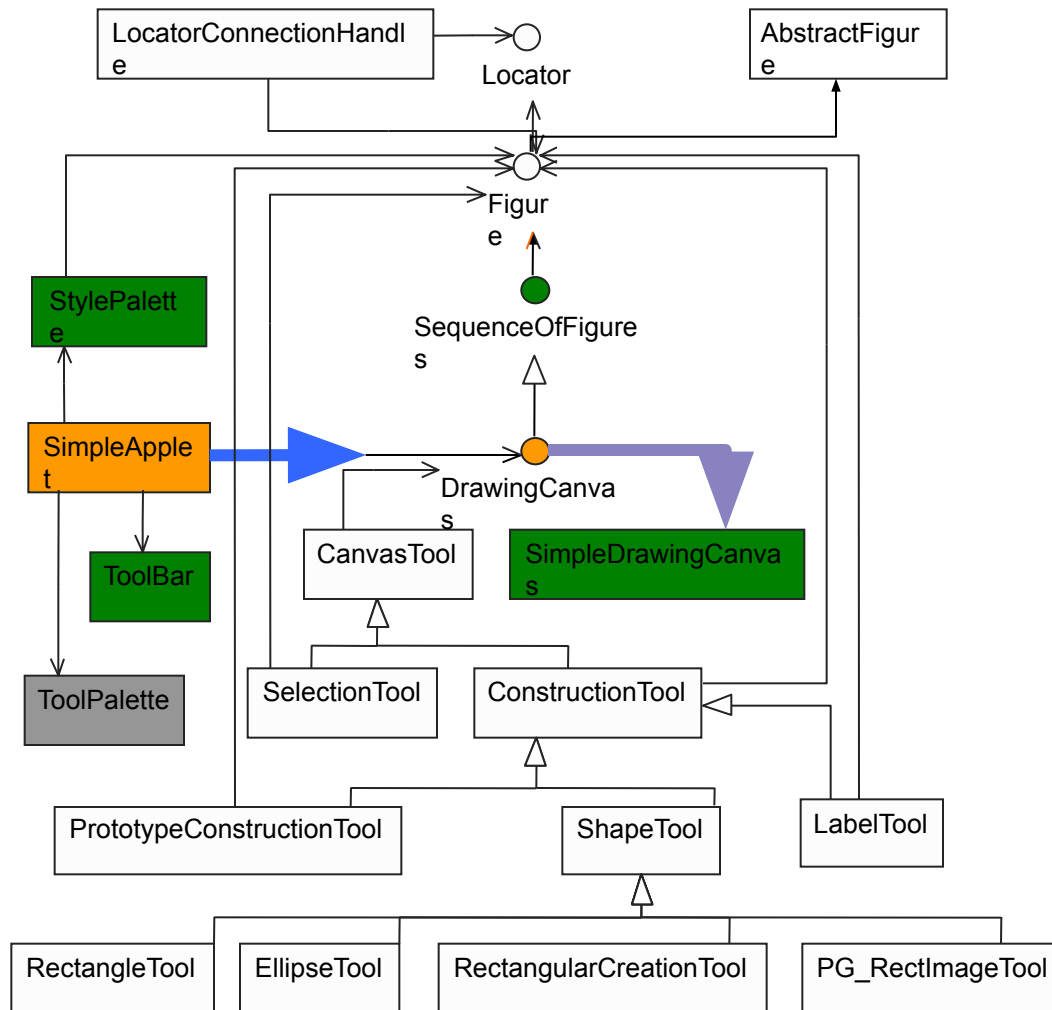
Backtrack



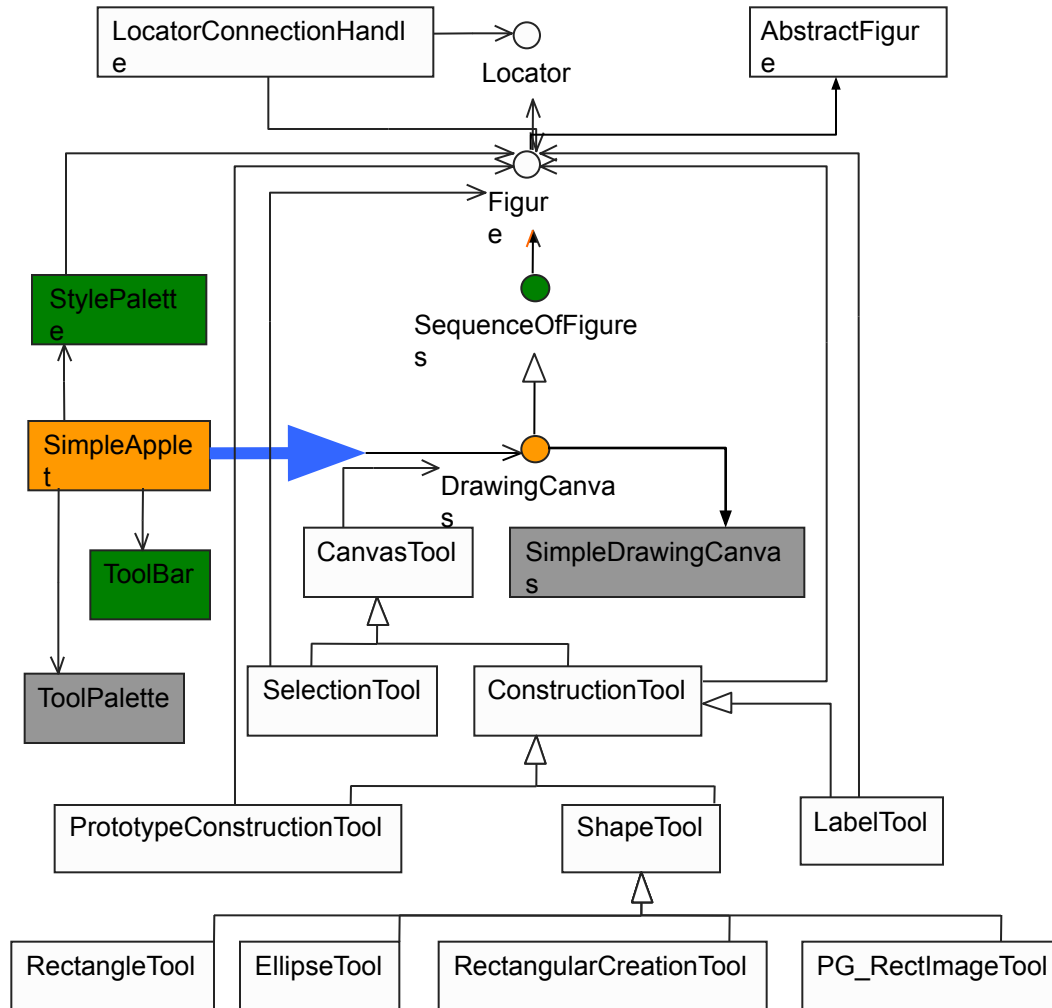
Right way



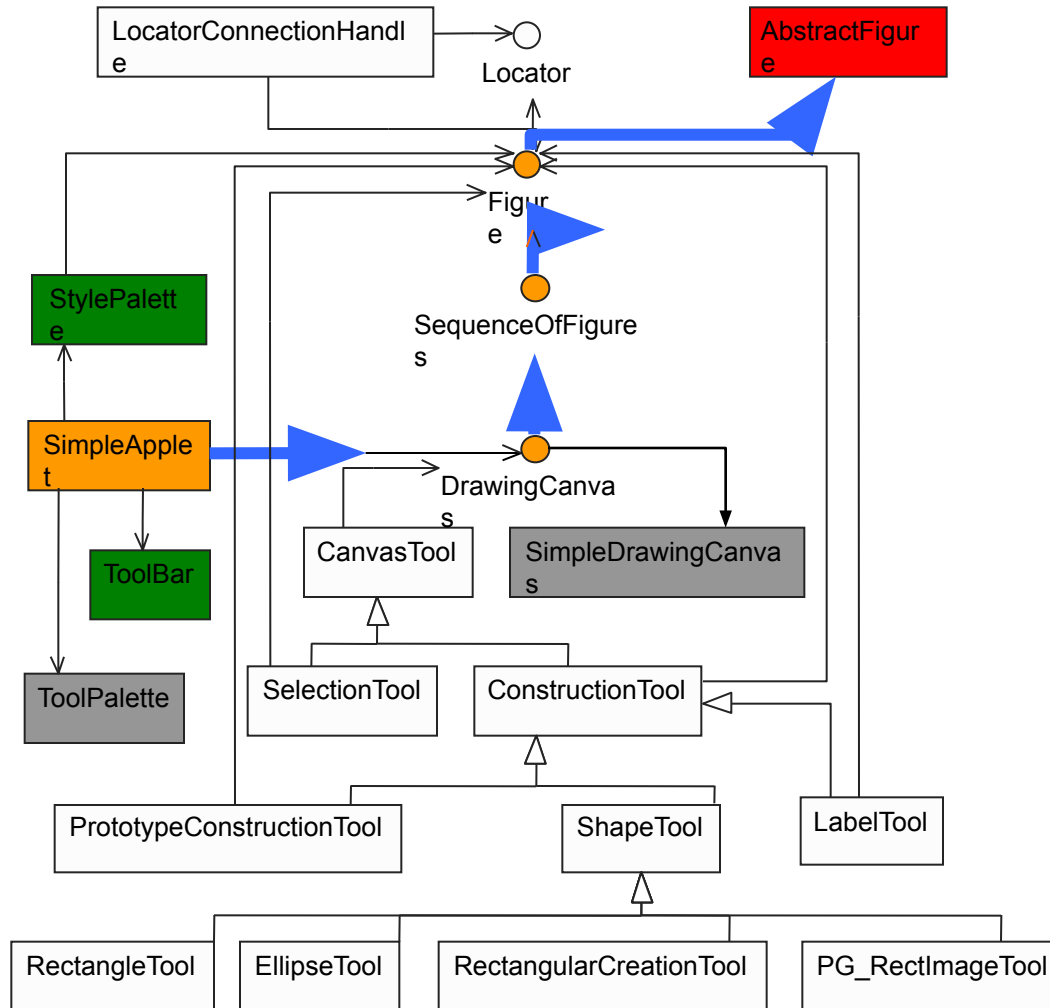
Wrong way



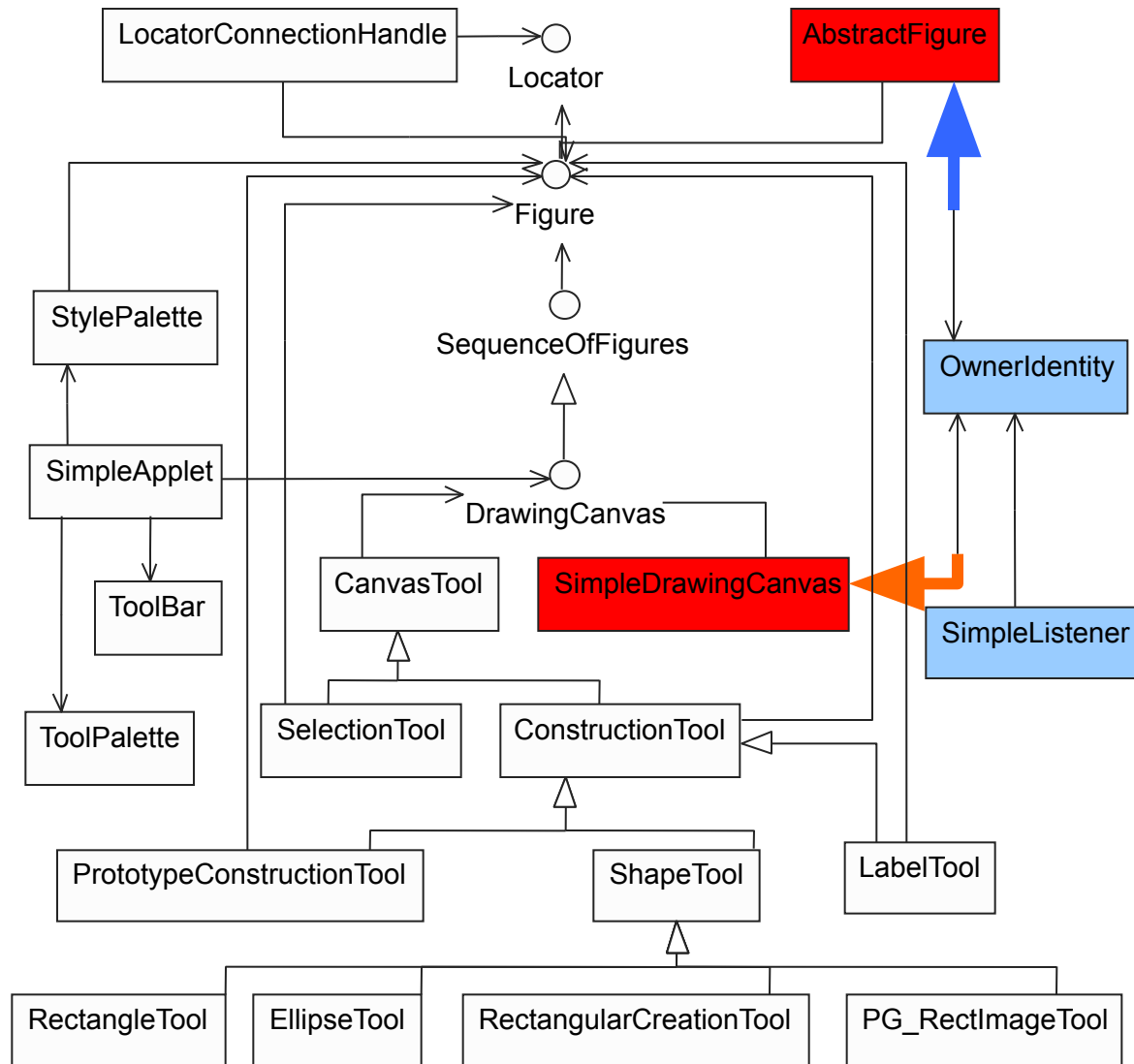
Backtrack



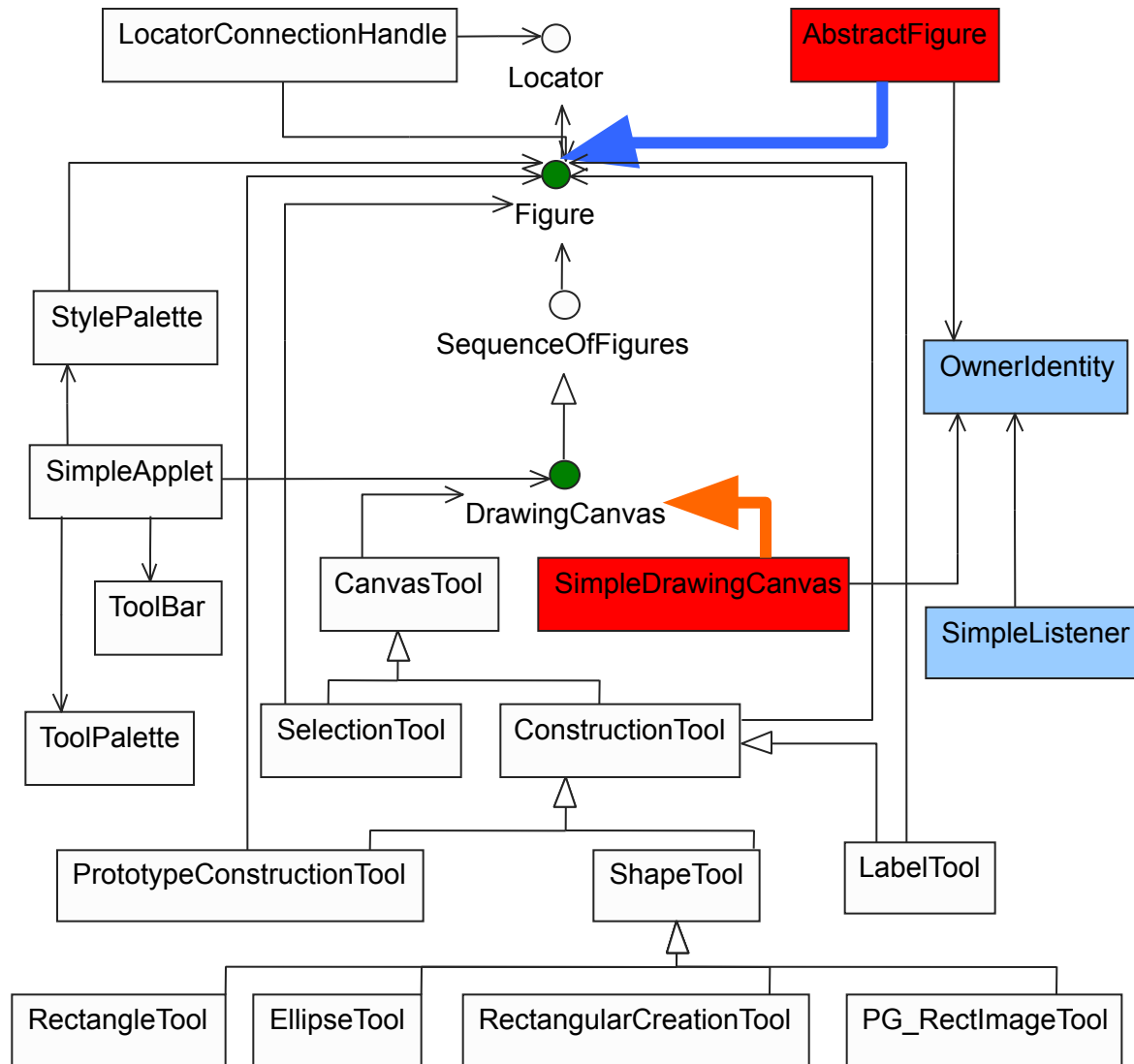
Summary



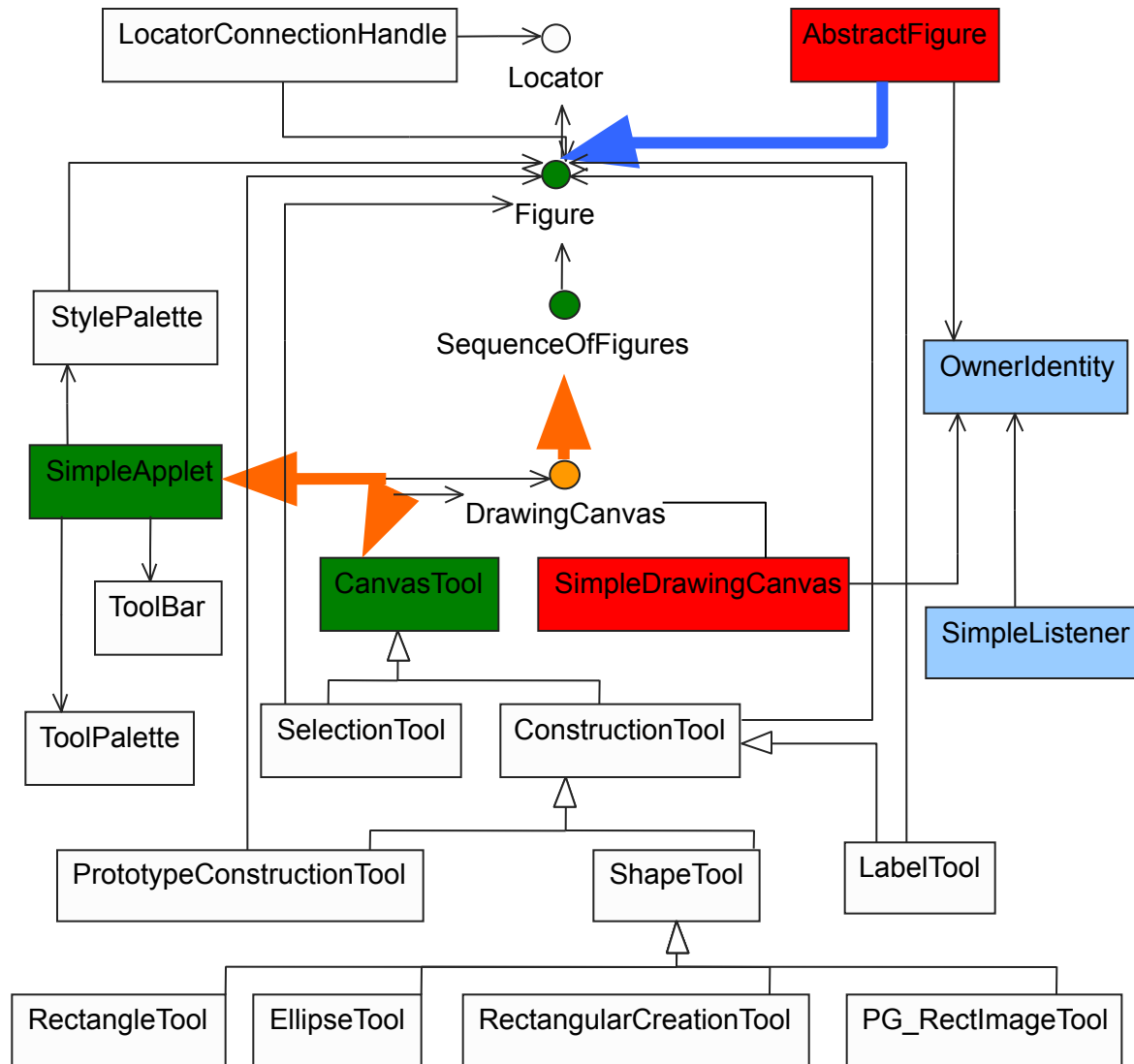
Actualization



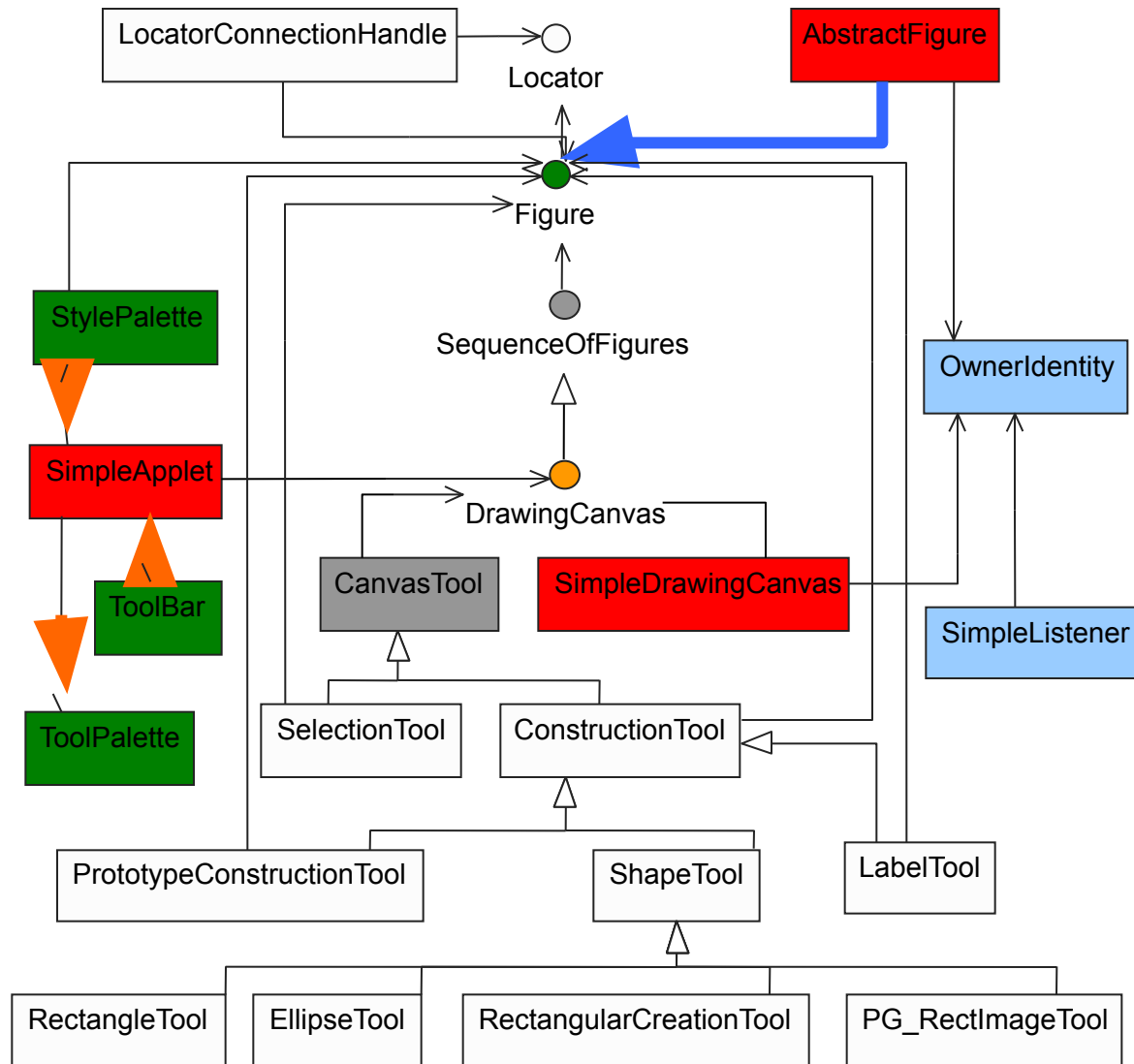
Change propagation



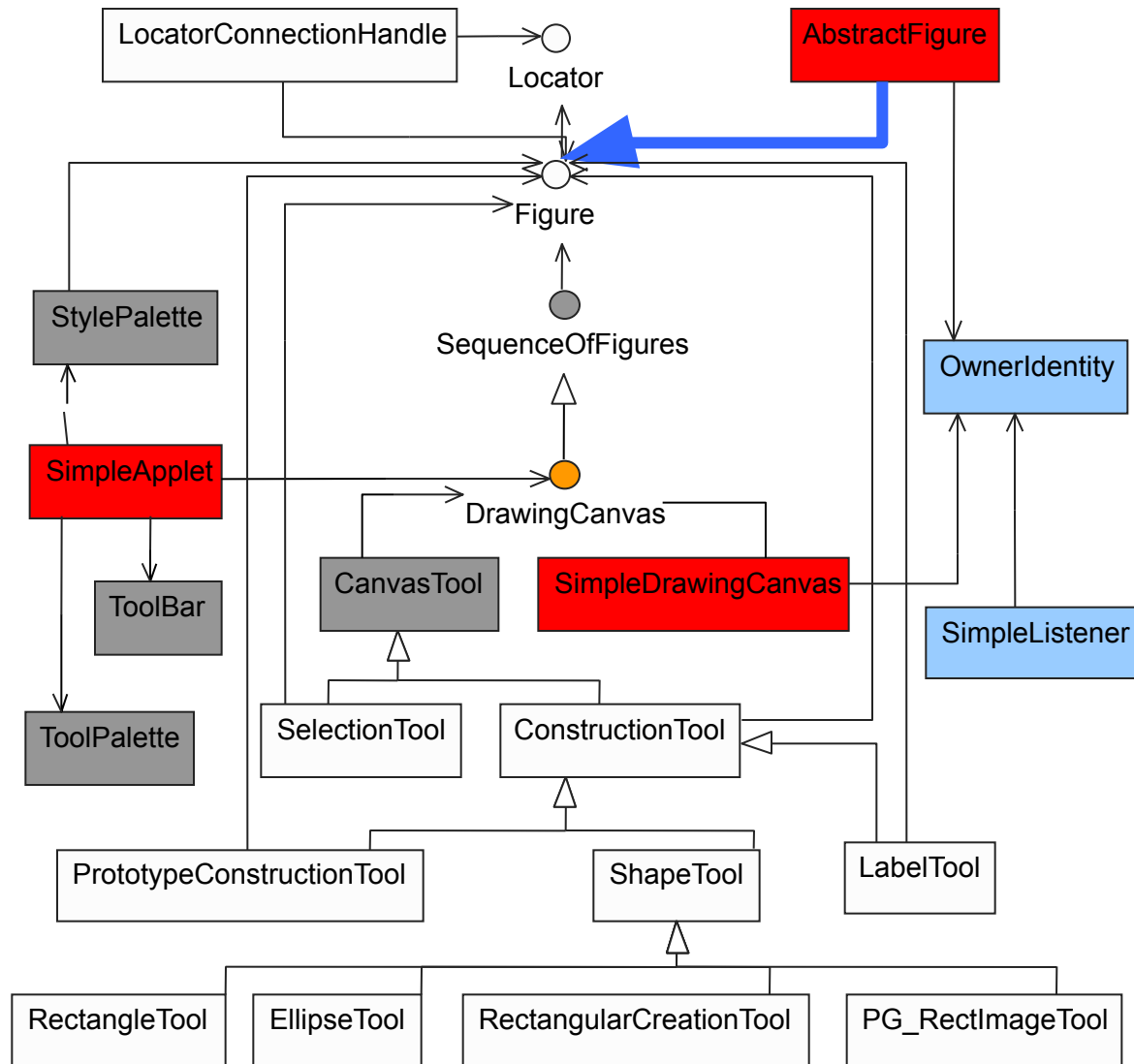
Propagation - 1



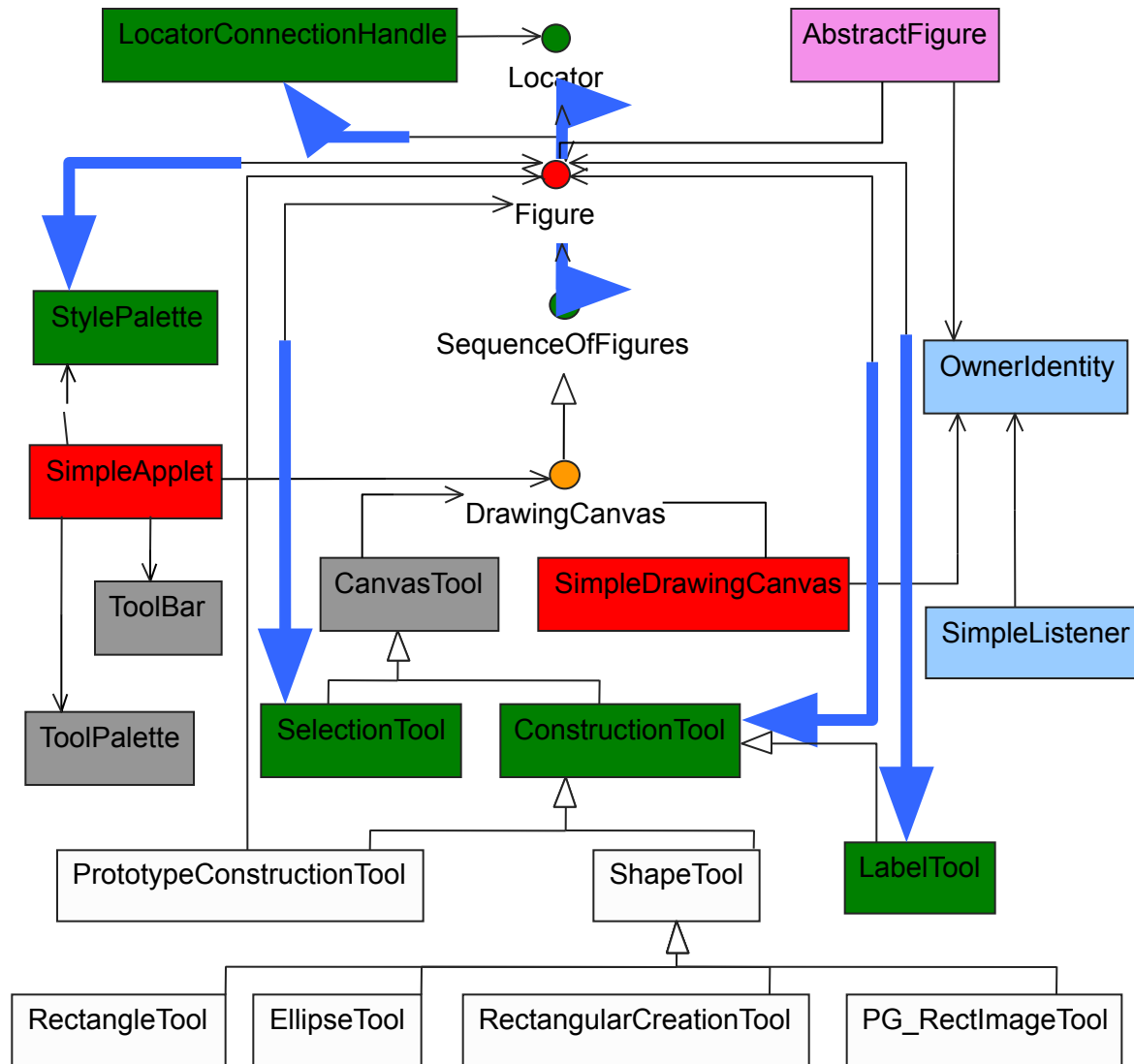
Propagation - 2



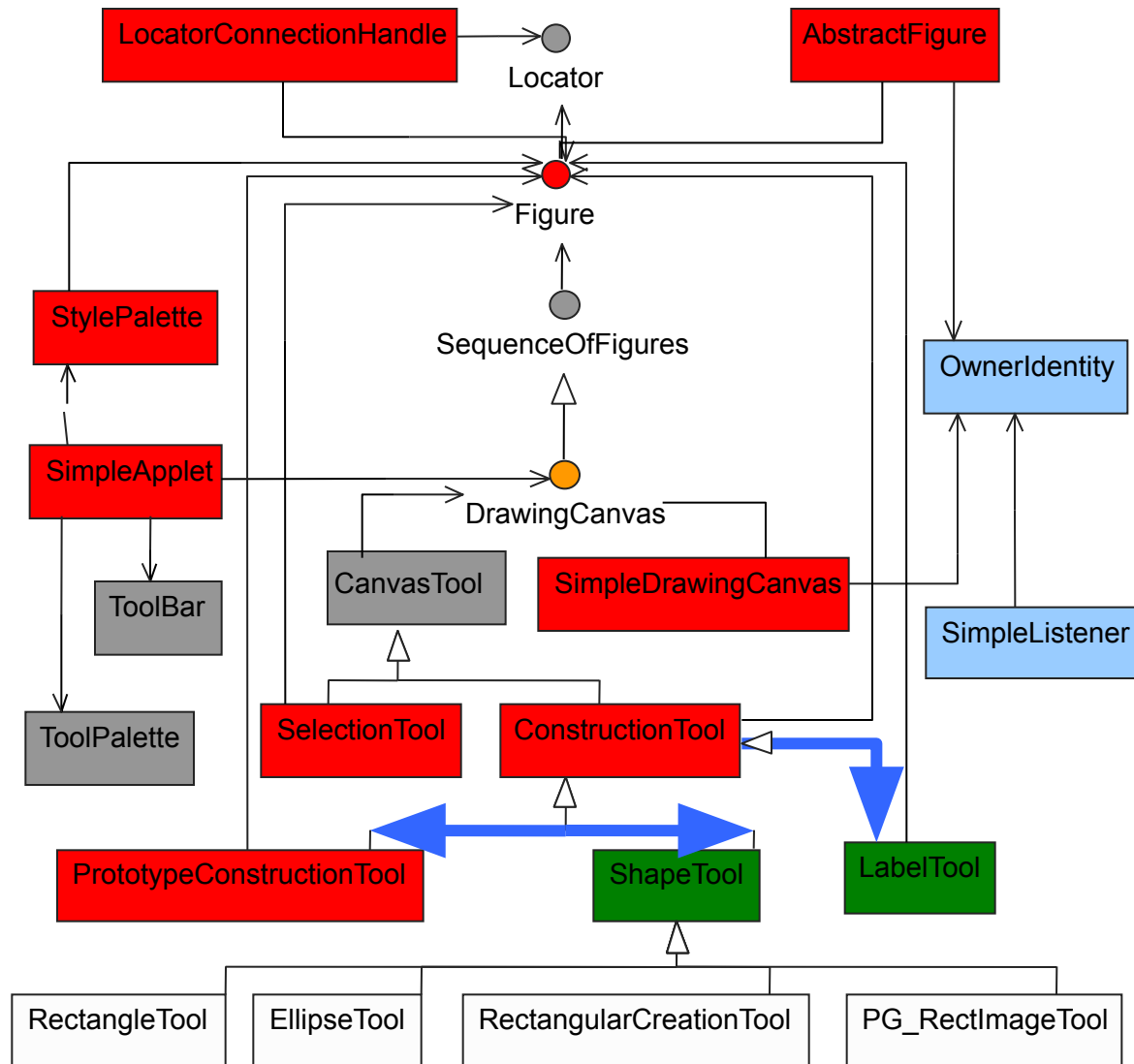
Propagation - 3



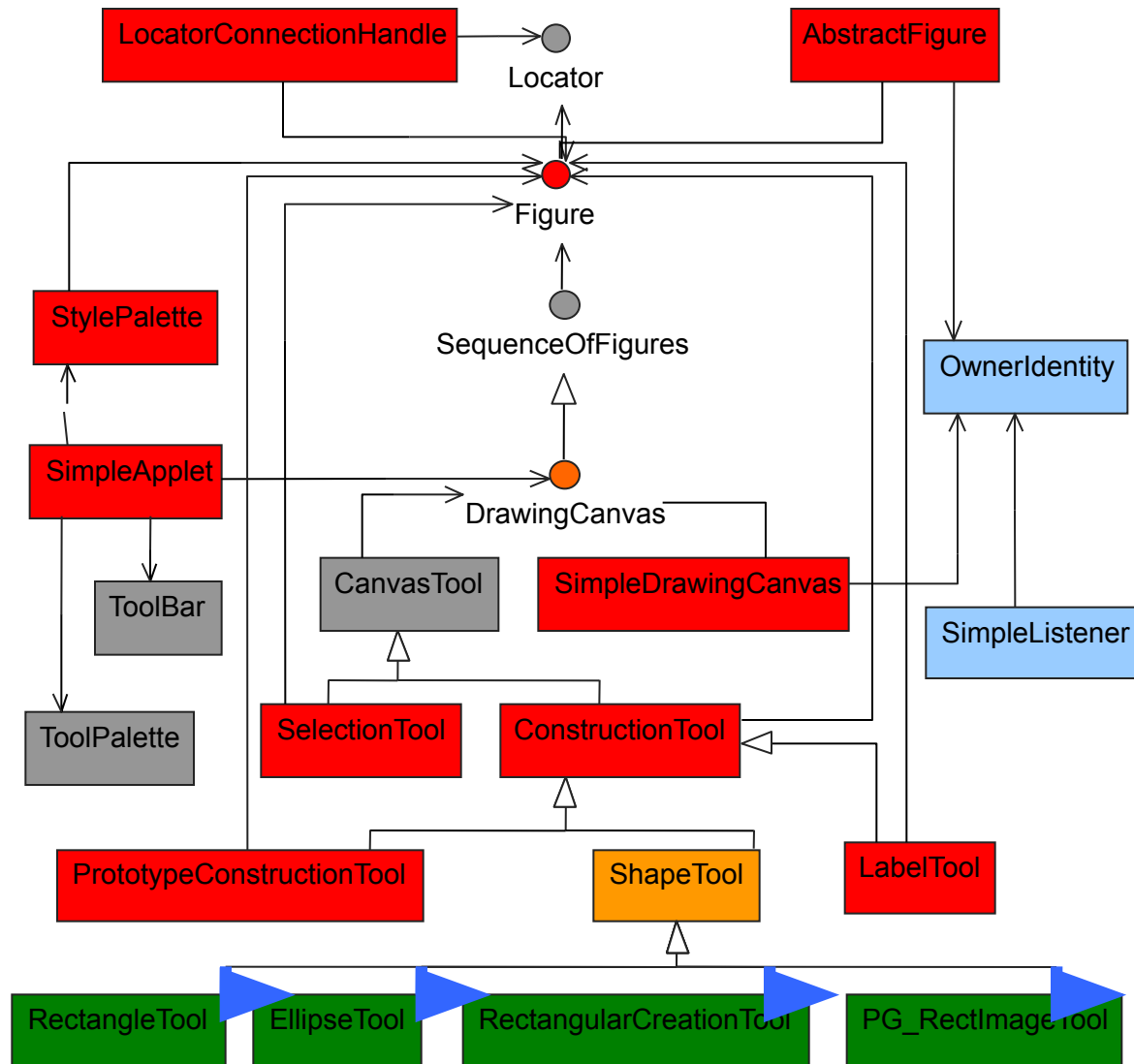
Propagation - 4



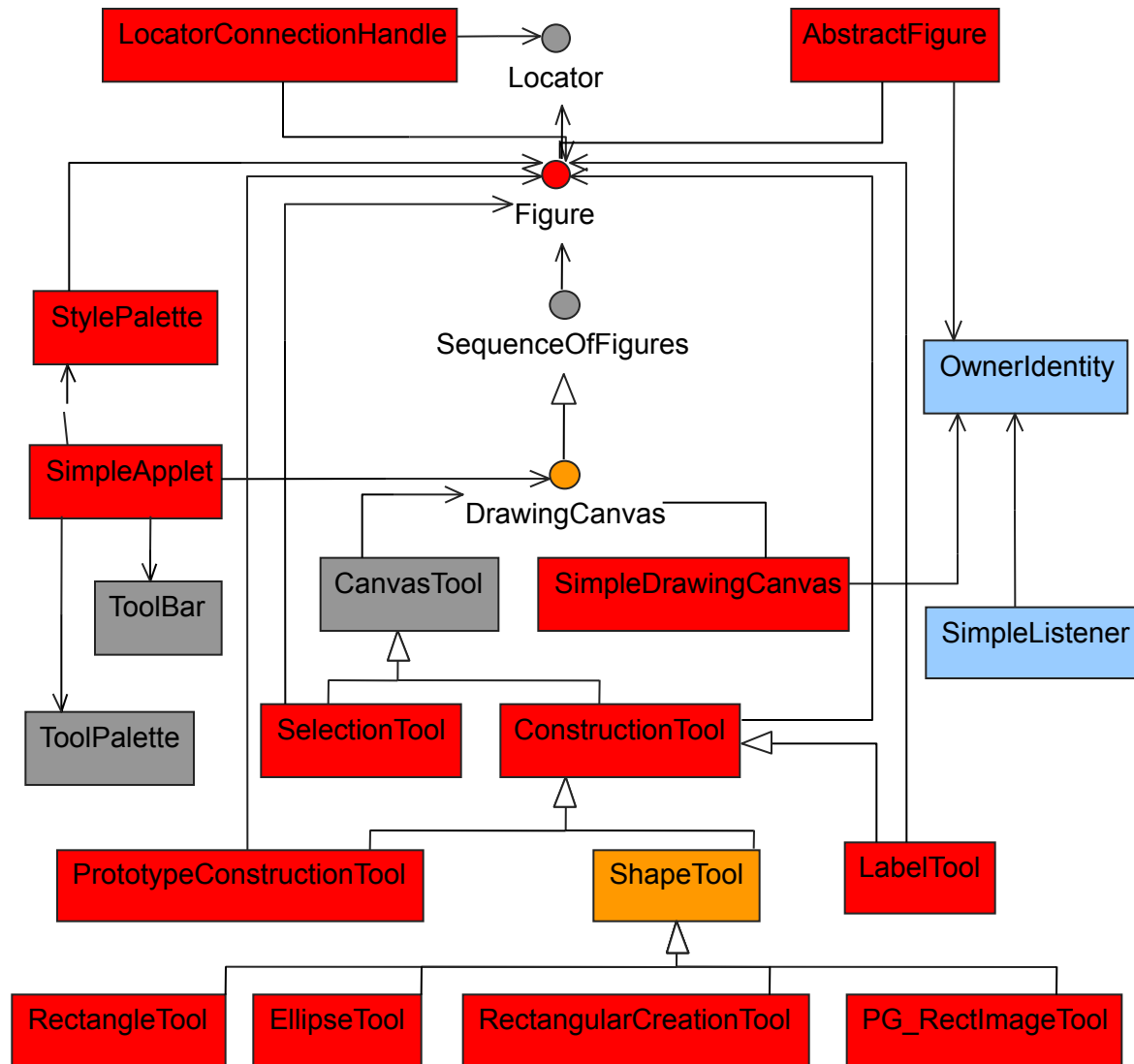
Propagation - 5



Propagation - 6



Propagation - done



Unit Tests

- 385 unit tests
- 1369 assertions
- 4800 lines.

Functional tests

- Draw
 - figure drawn on the canvas, for all figures
- Select
 - figure is selected
- Move
 - figure is moved to a different location
- . . .
 - 141 functional test cases

Creation of Acceptance tests

- Record/playback tool for Java applications
- Tool JGiven and Mockito used to run the functional tests

Phase of actualization

- New unit tests for new classes
- New functional tests for the new functionality
- During the change propagation, the old tests that were impacted by the change were updated.

Test suite maintenance

- Tests from the old version that are not affected by the change are kept as regression tests for the future
- Obsolete tests are removed
- Tests of the new features are added after the change

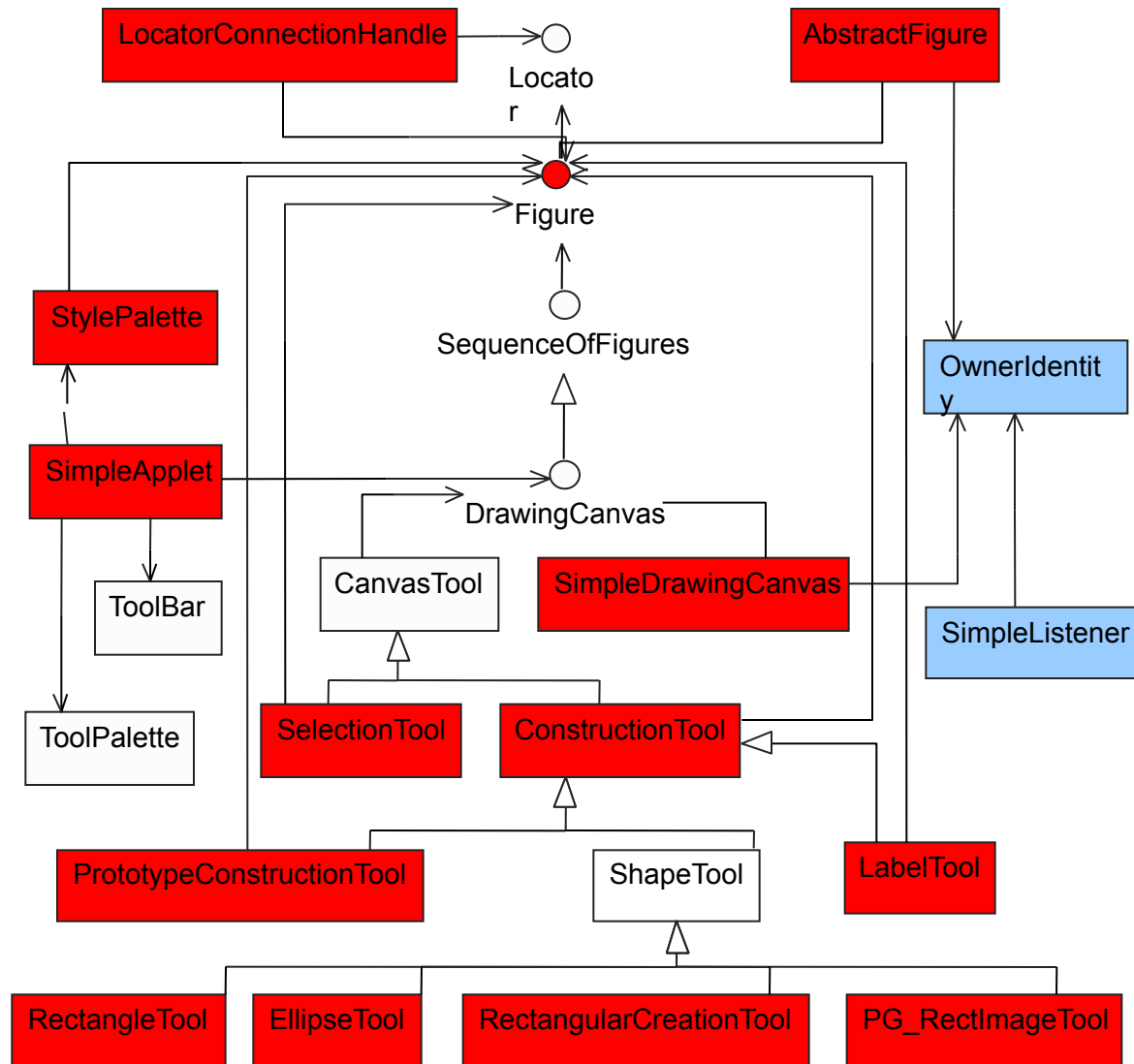
Results

- Base line
 - production code 17800 lines
 - unit test code 4800 lines.
- Change
 - 91 lines of production code (0.5%) were modified
 - 124 lines (2.5%) of the test code
 - approximately 1.4 test code lines per modified production code line.

Refactoring

- Long change propagation is a problem
 - it would be advantageous to shorten it
- Refactoring can shorten the change propagation
 - move code affected by change into fewer classes
 - split the roles

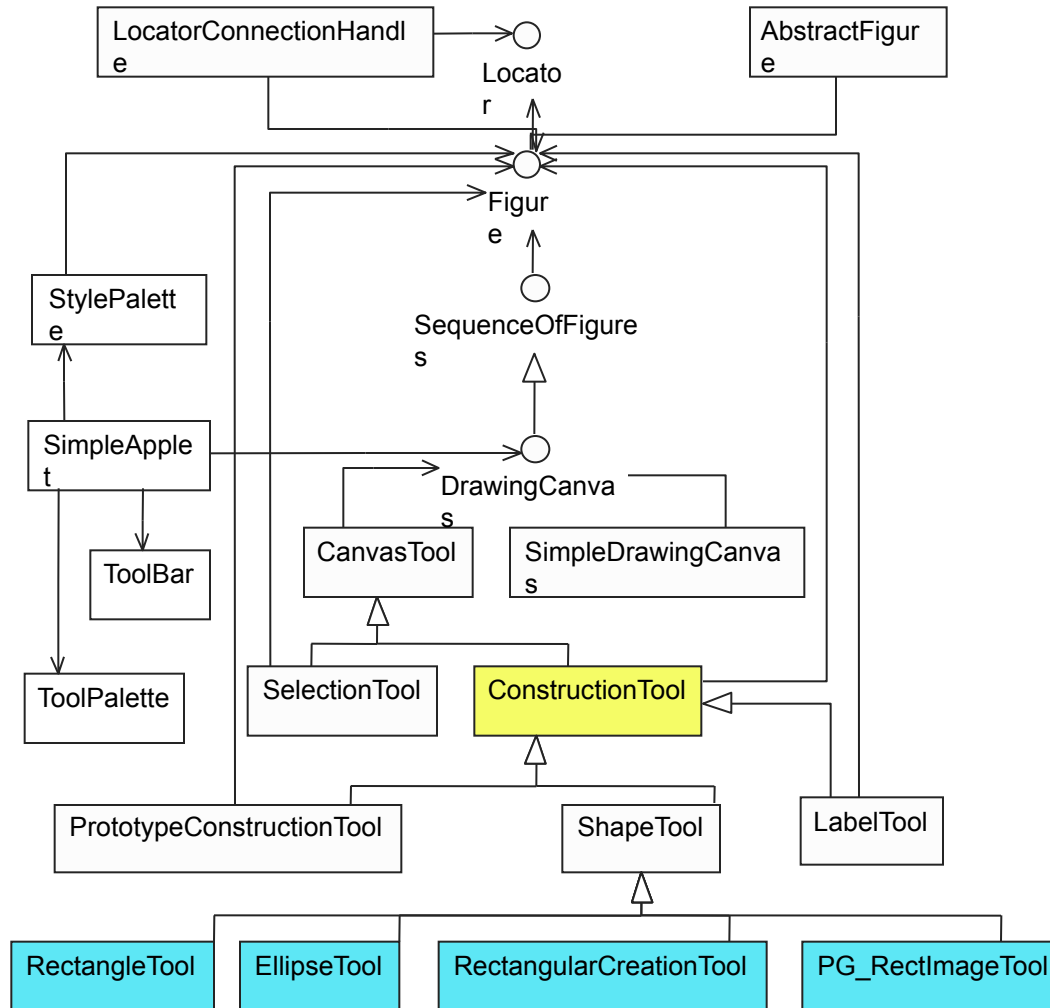
Propagation



Moving the code

- Each class that creates new figures has the function `basicNewFigure(...)`
 - two main parts:
 - create a new figure
 - if the figure was created at wrong location, move it
- We refactored a new method called `moveFigure(...)`
 - made it a member of the base class `ConstructionTool`

Refactoring impact



Splitting of roles

- Same method used in two different roles
 - the same code can do both jobs
- Propagating change highlights the differences
 - only one of the roles needs to be updated
 - the other one can stay unchanged
 - splitting the two roles and updating only one of them shortens the propagation

Splitting the roles

- Function `move(...)` in `AbstractFigure` is used in two ways
 - to move the figure as requested by the user
 - must check user identity
 - as a part of the new figure creation
 - does not need to check user identity
- We split `move(...)` and `secureMove(...)`

Numerical data

	No refactoring	Move function	Splitting
Classes added	2	2	2
Interfaces modified	1	1	1
Classes modified	13	8	5
LOC modified	91	95	87

Conclusions

- Splitting up a method into two similar methods may require least work
- Minus: A new set of unit tests must be created, thus test code is duplicated.
- Minus: More effort is required to create new tests compared to the effort required to adapt existing tests to their changed implementation.